

Optimization Algorithm

Optimization Algorithms

- Optimization algorithms helps us to *minimize (or maximize)* an **Objective** function (*another name for Error function*) $E(\mathbf{x})$ which is simply a mathematical function dependent on the Model's internal **learnable parameters** which are used in computing the target values($\mathbf{Y}$) from the set of *predictors*($\mathbf{X}$) used in the model.
- For example — we call the **Weights(W)** and the **Bias(b)** values of the neural network as its internal learnable *parameters* which are used in computing the output values and are learned and updated in the direction of optimal solution i.e minimizing the **Loss** by the network's training process and also play a major role in the *training* process of the Neural Network Model .

Optimization Algorithms

- Gradient Descent
- Gradient Descent with momentum
- RMSprop
- Adam Optimization algorithm

Gradient Descent

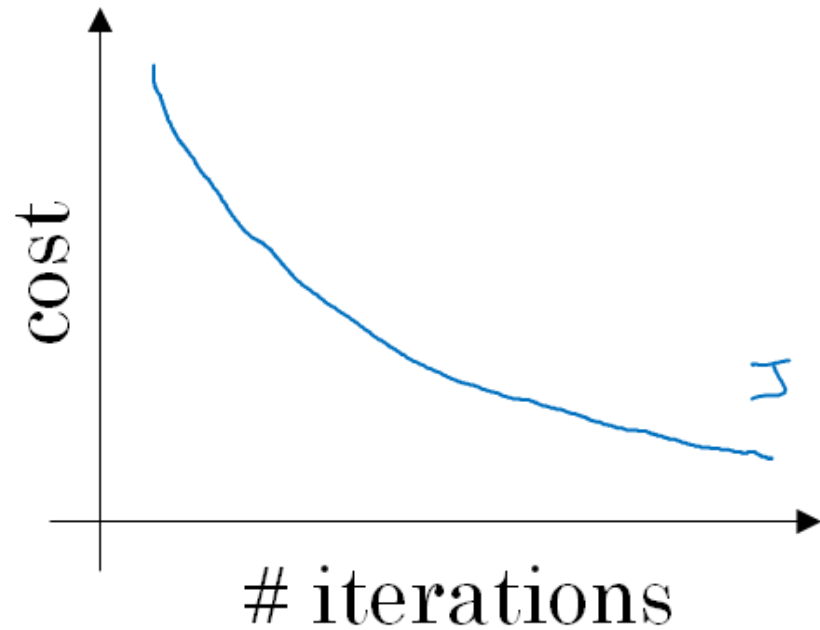
- Batch gradient descent
- mini-batch gradient descent
- Stochastic gradient descent

Mini-batch gradient descent

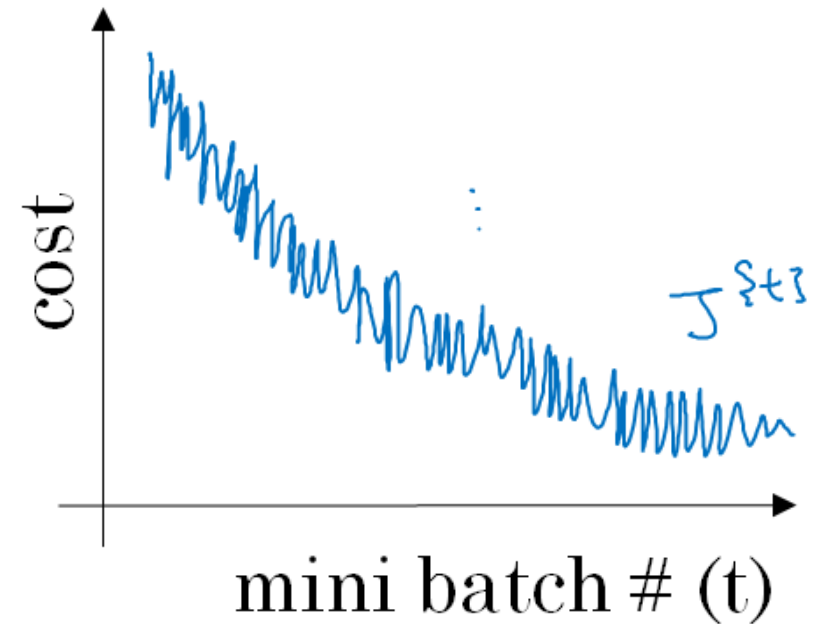
- Usually applied to training sets with large number of examples
- Take a small batch of training examples
- Compute loss on the current batch
- Compute gradients based on the loss of current batch
- Update learnable parameters
- 1 pass through all training examples is called one epoch

Training with mini-batch gradient descent

Batch gradient descent

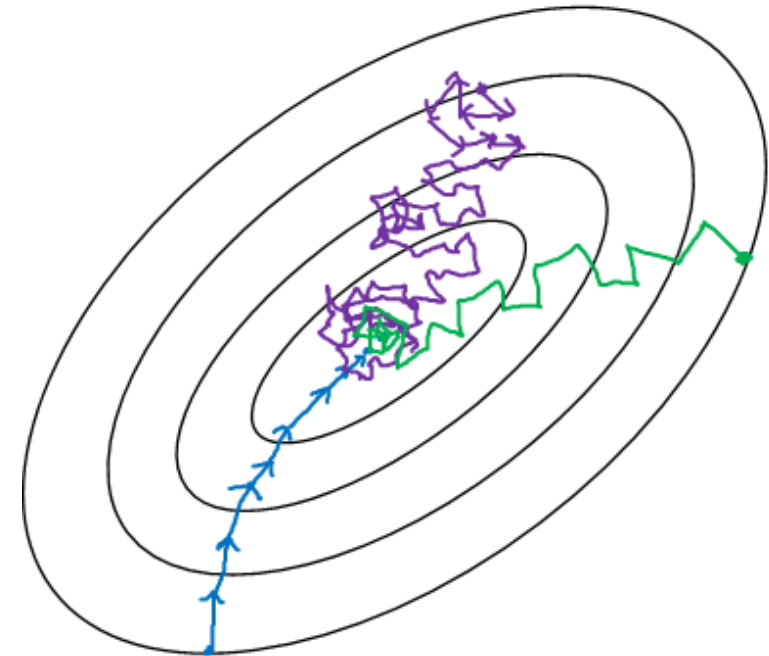
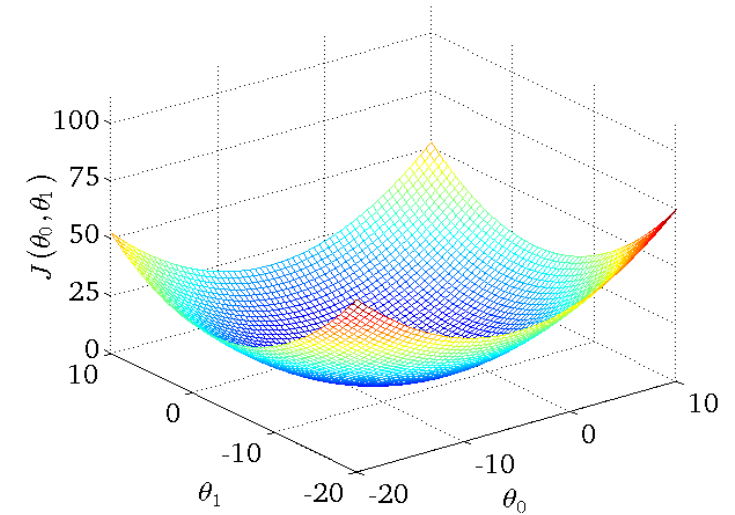


Mini-batch gradient descent



How to choose mini-batch size?

- Batch gradient descent: mini-batch size = m
 - Stochastic gradient descent: mini-batch size = 1
 - Mini-batch gradient descent: $1 < \text{mini-batch size} < m$
- If less training examples (e.g., ≤ 2000),
use Batch GD
 - Typical Batch-sizes: 64, 128, ..., 512
 - Make sure it fits in CPU/GPU memory



Exponentially Weighted Average

- A fast and efficient way to compute moving averages - implemented in the different optimization algorithms

Example: Temperature θ_t over days, calculate the moving averages

v_t : Moving average value at day 't'

$$v_0 = 0$$

$$v_1 = 0.9v_0 + 0.1\theta_1$$

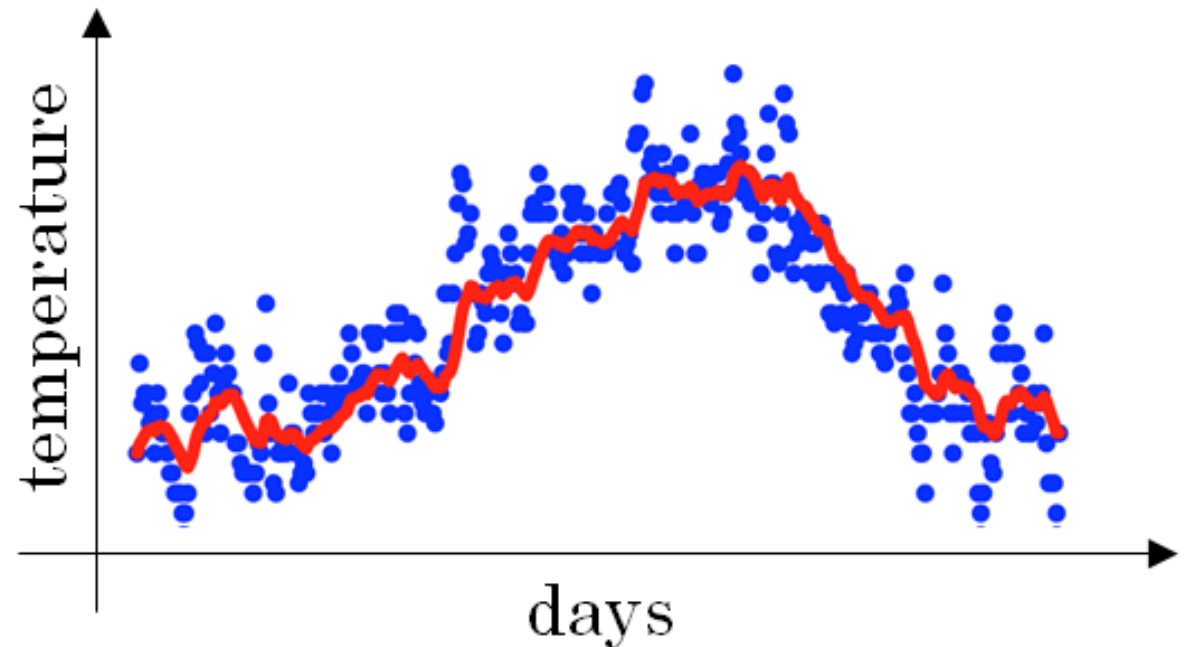
$$v_1 = 0.9v_1 + 0.1\theta_2$$

..

$$v_t = 0.9v_{t-1} + 0.1\theta_t$$

if $\beta = 0.9$

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$



Exponentially Weighted Average

For ex. , For $\beta=0.9$, $\frac{1}{1-\beta} \sim 10$;

$\beta = 0.9$ averages over 10 days (smooth curve: Red Line)

For $\beta=0.98$, $\frac{1}{1-\beta} \sim 50$;

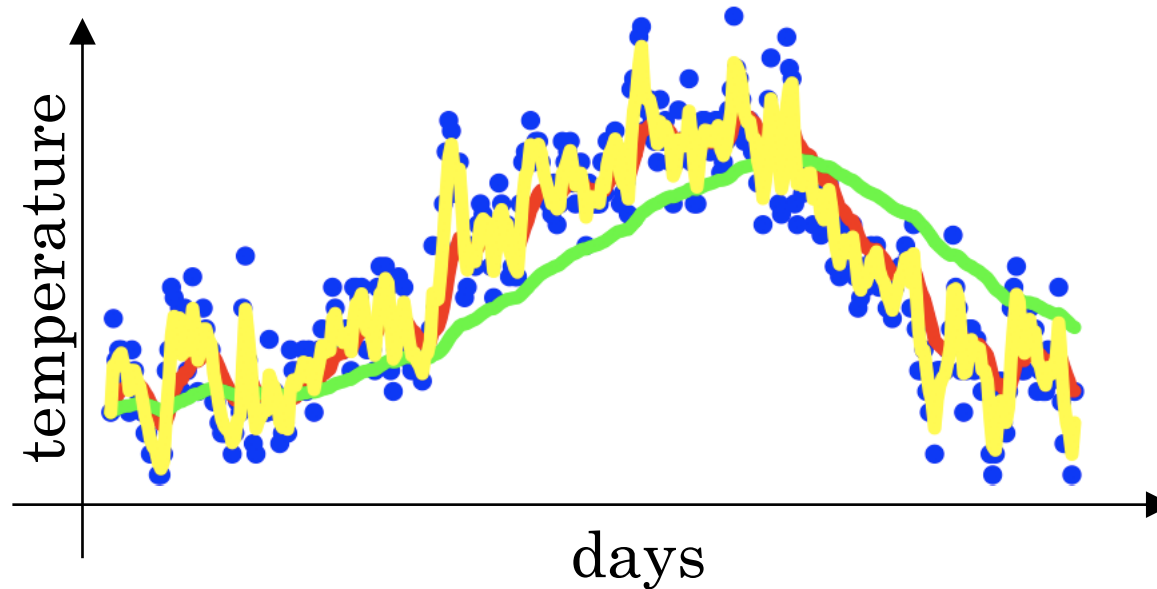
$\beta = 0.98$ averages over 50 days (smoother curve: Green Line)

For $\beta=0.5$, $\frac{1}{1-\beta} \sim 2$;

$\beta = 0.5$ averages over 2 days (Fluctuations: Yellow Line) - Much more noisy

What does v_t and β means

v_t : averaging over $\frac{1}{1-\beta}$ days (approx)



What is Exponentially weighted average actually doing?

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

Going backwards from v_{100} ,

$$v_{100} = 0.1\theta_{100} + 0.9v_{99}$$

$$v_{99} = 0.1\theta_{99} + 0.9v_{98}$$

Substituting v_{99} ,

$$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98})$$

$$\text{or, } v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97}))$$

Generalizing,

$$v_{100} = 0.1\theta_{100} + 0.1 * 0.9 * \theta_{99} + 0.1 * (0.9)^2 \theta_{98} + 0.1 * (0.9)^3 \theta_{97} + + 0.1 * (0.9)^4 \theta_{96} + \dots$$

v_{100} is basically an element wise computation of two metrics/functions - one an exponential decay function containing diminishing values ($0.9, 0.9^2, 0.9^3, \dots$) and another with all the elements of θ_t .

What is Exponentially weighted average actually doing?

Also, if $\beta = 0.9$, the weight decays to about a third by 10th iteration.

Proof: $(1 - \epsilon)^{\frac{1}{\epsilon}} = \frac{1}{e}$

$$\epsilon = 1 - \beta$$

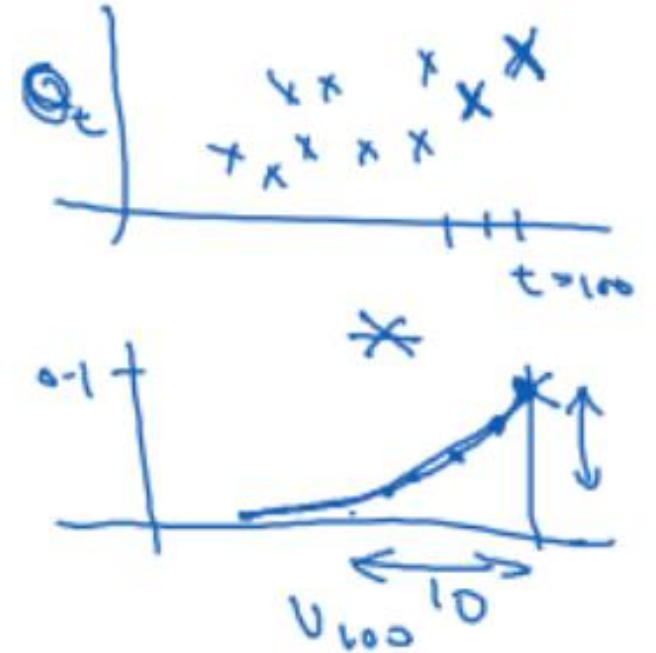
of $\beta = 0.9$, $\epsilon = 1 - \beta = 0.1$

$$(1 - \epsilon)^{\frac{1}{\epsilon}} = 0.9^{10} = 0.35 = \frac{1}{e}$$

Interpretation: It takes about 10 days for height to decay to 1/3rd

If $\beta = 0.98$, $\epsilon = 0.02$, $\frac{1}{\epsilon} = 50$;

It takes approx 50 days for height to decay to 1/3rd



Implementing Exponentially Weighted Average

v_θ : v is computing exponentially weighted average of parameter θ .

day 0: $v_\theta = 0$ day 1: $v_\theta = \beta v + (1 - \beta)\theta_1$

day 2: $v_\theta = \beta v + (1 - \beta)\theta_2$

...

Algorithms: $v_\theta = 0$ Repeat: {

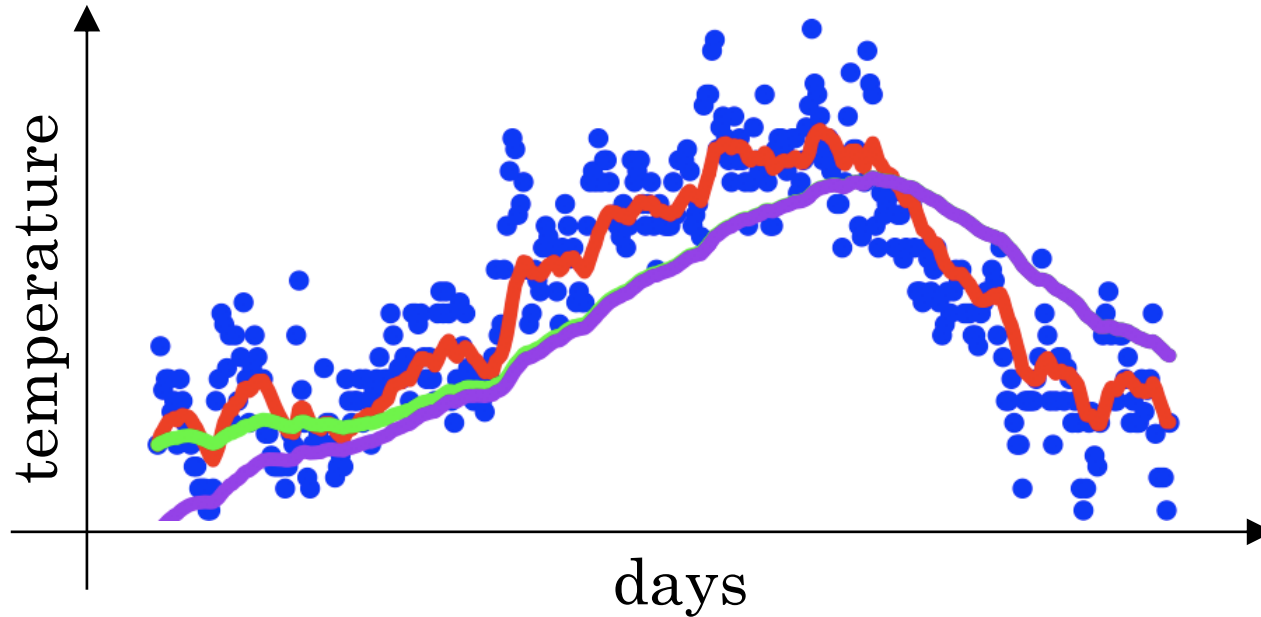
Get next θ_t

$v_\theta := \beta v_\theta + (1 - \beta)\theta_t$

}

Single line implementation for fast and efficient calculation of exponentially weighted moving average.

Bias correction in Exponentially Weighted Moving Average



$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

Figure: The ideal curve should be the GREEN one, but it starts as the PURPLE curve since the values initially are zero

Bias correction in Exponentially Weighted Moving Average

Example: Starting from $t=0$ and moving forward,

$$v_0 = 0 \quad v_1 = 0.98v_0 + 0.02\theta_1 = 0.02\theta_1$$

$$v_2 = 0.98v_1 + 0.02\theta_2 = 0.0196\theta_1 + 0.02\theta_2$$

The initial values of v_t will be very low which need to be compensated.

$$\text{Make } v_t = \frac{v_t}{1-\beta^t}$$

$$\text{for } t=2, 1 - \beta^t = 1 - 0.98^2 = 0.0396 \text{ (Bias Correction Factor)}$$

$$v_2 = \frac{v_2}{0.0396} = \frac{0.0196\theta_1 + 0.02\theta_2}{0.0396}$$

When t is large, $\frac{1}{1-\beta^t} = 1$, hence bias correction factor has no effect when t is sufficiently large. It only jacks up the initial values.